

Teste de Software – Test Smell

1. Análise de Smells

A suíte de testes original apresentava diversas práticas não recomendadas no contexto de Qualidade de Software, conhecidas como Test Smells. Esses test smells não impedem a execução dos testes, mas degradam a manutenibilidade, a confiabilidade e a clareza do projeto a longo prazo. Durante a análise, destacaram-se três smells principais:

1.1. Eager Test (Teste Guloso)

- **Onde foi encontrado:** No teste nomeado deve criar e buscar um usuário corretamente.
- **Porque é um smell:** Este teste tentava validar duas funcionalidades distintas no mesmo fluxo de execução: a criação de um registro e, logo em seguida, a busca desse mesmo registro. Ele viola o Princípio da Responsabilidade Única (SRP) aplicado a testes, que dita que cada teste deve ter apenas um motivo para falhar.
- **Risco associado:** A principal consequência do Eager Test é a dificuldade de diagnóstico. Se o teste falhar na etapa de busca, o relatório de erro não deixará claro imediatamente se o problema está no algoritmo de recuperação de dados ou se a falha ocorreu silenciosamente na etapa anterior de criação (gravando o dado incorretamente). Isso aumenta o tempo de depuração.

1.2. Conditional Test Logic (Lógica Condicional em Testes)

- **Onde foi encontrado:** No teste deve desativar usuários se eles não forem administradores.
- **Porque é um smell:** A presença de estruturas de controle de fluxo, como for e if/else, dentro do escopo de asserção. Testes devem ser lineares e determinísticos (padrão de passos fixos). Adicionar lógica de programação complexa transforma o código de teste em algo que, paradoxalmente, também precisa ser testado.
- **Risco associado:** A lógica condicional torna os testes não confiáveis. Se a condição lógica do if possuir um bug ou os dados gerados não satisfizerem a premissa do laço de repetição, asserções (expects) cruciais podem ser simplesmente ignoradas pela engine do Jest sem acusar erro. Isso gera falsos positivos, mascarando falhas reais no código de produção.

1.3. Fragile Test (Teste Frágil / Sensibilidade Excessiva)

- **Onde foi encontrado:** No teste deve gerar um relatório de usuários formatado.
- **Porque é um smell:** O teste acoplava sua verificação à estrutura exata de uma string gerada pelo sistema (ex: checando espaços, quebras de linha e pontuações específicas).
- **Risco associado:** Testes frágeis quebram não porque o comportamento do sistema falhou, mas porque detalhes irrelevantes de apresentação mudaram. Uma simples refatoração estética no relatório (como adicionar um espaço ou mudar uma vírgula) faria a suíte falhar. Isso gera o fenômeno da “fadiga de testes”, onde a equipe passa mais tempo consertando testes quebrados do que desenvolvendo novas features, o que muitas vezes leva ao abandono da cultura de testes.

2. Processo de Refatoração

Para ilustrar o processo de limpeza e adequação aos padrões de engenharia de software, focamos na remoção do Conditional Test Logic. A refatoração foi baseada na aplicação rigorosa do padrão **AAA (Arrange, Act, Assert)** e na divisão de responsabilidades.

O Antes (Código com Smells)

No código original, a validação ocorria dentro de um laço de repetição com múltiplas asserções dependentes de um if/else, tornando a leitura obscura e o escopo de falha amplo:

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  // O teste tem um loop e um if, tornando-o complexo e menos claro.
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      // Este expect só roda para o usuário comum.
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      // E este só roda para o admin.
      expect(resultado).toBe(false);
    }
  }
});
```

O Depois (Código Clean)

A solução adotada foi dividir o comportamento complexo em dois testes independentes, focados e sem controle de fluxo. Cada teste agora narra uma única "história" clara.

```
test('deve desativar um usuário comum com sucesso', () => {
  // Arrange
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  expect(usuarioAtualizado.status).toBe('inativo');
});

test('não deve permitir a desativação de um usuário administrador', () => {
  // Arrange
  const isAdmin = true;
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, isAdmin);

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);

  // Assert
  expect(resultado).toBe(false);
});
```

Decisões Tomadas:

1. **Divisão de Cenários:** O teste original misturava o "caminho feliz" (usuário comum) e a "restrição de negócio" (administrador). Ao dividi-los, os títulos dos testes tornaram-se a própria documentação da regra de negócio.
2. **Padrão AAA:** O código foi visualmente dividido com comentários (Arrange, Act, Assert). Isso calibra o modelo mental de qualquer desenvolvedor que realizar a manutenção do repositório no futuro, isolando a preparação dos dados da ação executada.
3. **Remoção de Iteração:** Removendo o for, garantimos que o teste seja executado de forma linear. Se o teste de administrador falhar, o de usuário comum ainda executará com sucesso e passará, facilitando a identificação exata do bug.

3. Relatório da Ferramenta de Análise Estática

Ferramentas de análise estática como o ESLint são a primeira linha de defesa contra a degradação estrutural do código. Abaixo, constam os apontamentos levantados pela ferramenta antes da nossa refatoração.

```
vinic@VXPC MSYS ~/OneDrive/Desktop/Teste de Software/test-smelly (main)
$ npx eslint .

C:\Users\vinic\OneDrive\Desktop\Teste de Software\test-smelly\test\userService.smelly.test.js
 44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning  Tests should not be skipped            jest/no-disabled-tests
 77:3  warning  Test has no assertions                 jest/expect-expect

X 6 problems (4 errors, 2 warnings)
```

A automação provida pelo ESLint (especialmente quando integrado com plugins voltados para testes, como eslint-plugin-jest) é vital. A ferramenta age de forma determinística e proativa, evidenciando de imediato as violações sintáticas e lógicas, por exemplo, o uso inadequado de laços de repetição em asserções (no-conditional-expect).

Em vez de depender apenas do raciocínio e do rigor manual durante as sessões de code review, o uso de linters impõe um padrão metodológico que bloqueia a submissão de artefatos com baixa qualidade antes mesmo que eles cheguem aos repositórios principais do projeto.

4. Conclusão

A avaliação sistemática da saúde dos repositórios de código revela que testes unitários são tão importantes quanto o próprio código de produção. A escrita de testes limpos supera a intuição pura do desenvolvedor; ela se apoia em métricas de qualidade sólidas para assegurar que a arquitetura do software tolere o tempo e as inevitáveis mudanças de requisitos.

Ao eliminar smells como testes gulosos e lógicas condicionais, nós não apenas melhoramos o tempo de execução e a precisão do diagnóstico, mas criamos um sistema imunológico robusto para a aplicação. Conclui-se que o rigor na aplicação de metodologias (como o padrão AAA) e a integração de ferramentas de análise estática contínua são indispensáveis. Eles transferem o custo cognitivo do

desenvolvedor para a automação, permitindo decisões de engenharia baseadas em evidências analíticas e promovendo a sustentabilidade de longo prazo do ciclo de desenvolvimento do software.

Após a refatoração o ESLint só encontrou os erros no arquivo antigo `userService.smelly.test.js` e não encontrou erros no `userService.clean.test.js`.

Todos os testes do comando “`npm test`” foram aprovados.

```
vinic@VXPC MSYS ~/OneDrive/Desktop/Teste de Software/test-smelly (main)
$ npx eslint .

C:\Users\vinic\OneDrive\Desktop\Teste de Software\test-smelly\test\userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)

vinic@VXPC MSYS ~/OneDrive/Desktop/Teste de Software/test-smelly (main)
$ npm test

> test-smells-lab@1.0.0 test
> jest

PASS test/userService.clean.test.js
PASS test/userService.smelly.test.js

Test Suites: 2 passed, 2 total
Tests:      1 skipped, 11 passed, 12 total
Snapshots:  0 total
Time:       0.379 s, estimated 1 s
Ran all test suites.
```